EMSI | Rabat

5IASD - Groupe G8 - Année universitaire 2026-2027

Compte rendu de TP05

Créer et déployer un modèle AutoML

Du modèle entraîné au service HTTP : registre MLflow, alias, consommation et comparaison avec une référence manuelle.

Réalisé par	Formation
Mohamed BENHEDDANE Yassir DRISSI Zineb BAQLOUL	5IASD - G8
Encadrante	Pr. H. Naciri
Date des mesures	02 novembre 2026

Identité du groupe reprise du rapport TP04 présent dans le même espace de travail.

Objectif et périmètre

Entraîner FLAML avec un budget de 90 secondes, enregistrer son gagnant et le Random Forest du TP2, puis servir leurs versions promues sur deux points de terminaison locaux. La comparaison porte sur la qualité, la latence, la taille et l'explicabilité.

Les blocs locaux sont exécutés. Le parcours cloud facultatif est exclu. Le dépôt Git est préparé localement ; la publication GitHub et l'envoi du rendu restent à effectuer après validation.

Adaptation au poste : registre sur 127.0.0.1:5005, car un serveur du TP4 utilise déjà 5000. Services de prédiction sur 8100 et 8101, conformément au sujet.

1. Protocole et résultats mesurés

Le générateur du TP4 produit 5 000 transactions synthétiques (graine 7), dont 568 fraudes. Les variables sont montant, anciennete, freq_24h et nuit ; la cible est fraude. La partition commune contient 3750 observations d'entraînement et 1250 de test, dont 134 fraudes.

Le découpage utilise test_size=0.25 et random_state=42, sans stratification, comme le sujet. FLAML choisit ses réglages sur son découpage interne d'entraînement/validation ; le test est réservé à l'évaluation finale. Deux fils de calcul bornent la recherche sur le poste.

Critère	Manuel RF 500/16	AutoML FLAML 90 s
Famille de modele	foret aleatoire (500 arbres, profondeur 16)	XGBClassifier
F1 sur le jeu de test	0.8669	0.8561
Precision / rappel	0.8837 / 0.8507	0.8692 / 0.8433
Temps machine (s)	2.86 (entrainement seul)	90.00 (recherche et reentrainement)
Temps humain	non mesure au TP2, a renseigner par le groupe	non mesure : mise en place et analyse a inclure
Latence mediane (ms), essais 1 / 2	92.13 / 118.10	31.21 / 30.34
Latence p95 (ms), essais 1 / 2	150.68 / 300.82	41.16 / 38.72
Taille du fichier (Mio)	13.374	0.279
Explicabilite	importance des variables, 500 arbres, profondeur max. 16	importance des variables, arbres et profondeur selon famille ; SHAP possible
Format dans MLflow	skops	cloudpickle

Mio = 1 048 576 octets. Les tailles portent sur les copies joblib, afin de comparer le même format. Le temps machine manuel exclut le réglage humain du TP2. Les durées humaines sont inconnues et doivent être précisées par le groupe.

Décision de production (5 lignes)

Je retiens le modele AutoML pour le compromis qualite, latence et taille observe dans ce TP.

Le F1 AutoML est 0.8561 contre 0.8669 ; il manque 21 fraudes contre 20 pour le manuel.

Ses medianes sont 31.21/30.34 ms contre 92.13/118.10 ms, et il est 47.9 fois plus compact.

Les deux modeles exposent des importances de variables ; je conserve le manuel comme reference et solution de retour arriere.

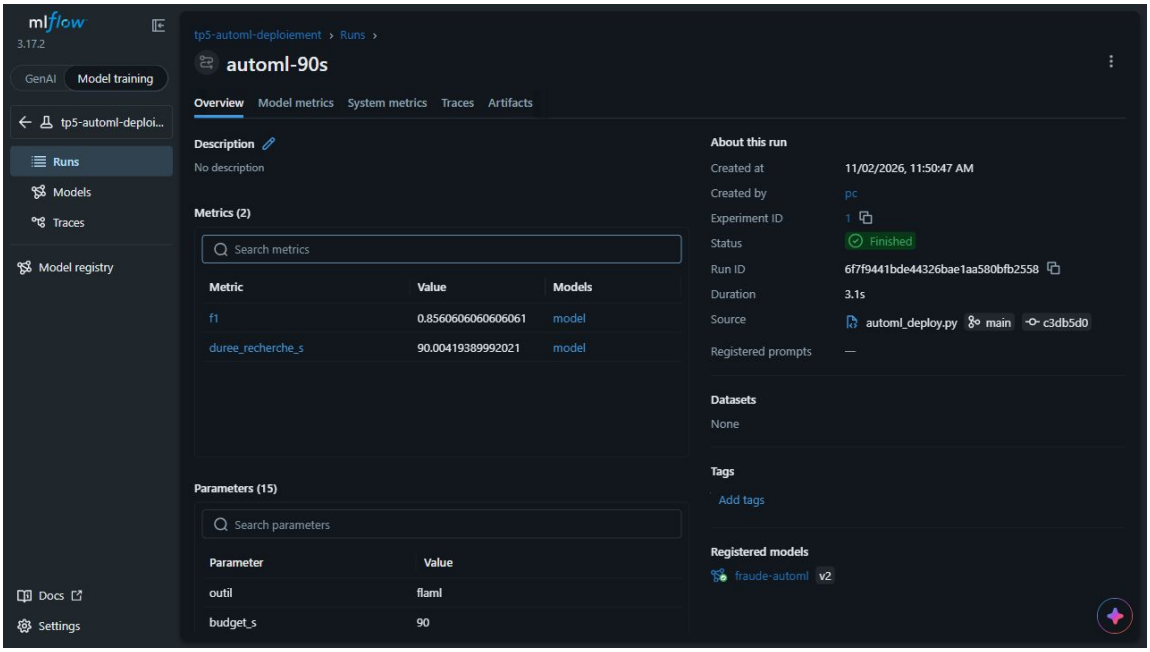
Ce choix pedagogique doit etre confirme avec un cout metier des faux negatifs, d'autres partitions et un objectif de latence explicite.

2. Entraînement, export et registre

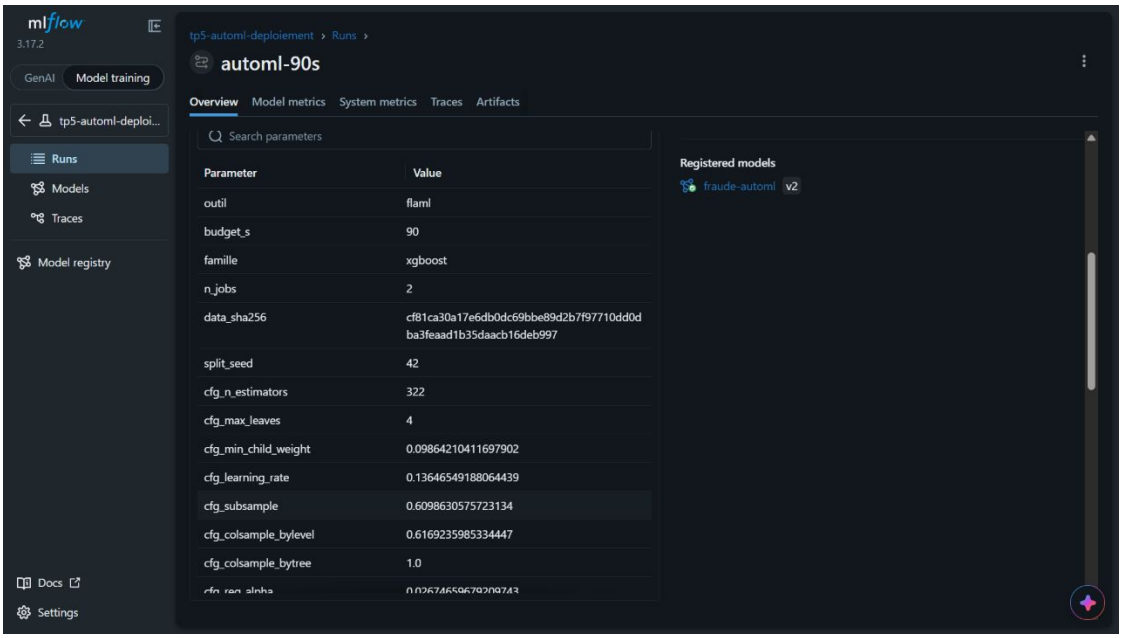
FLAML retient **XGBClassifier** (famille xgboost) après 90.00 s. Le F1 de l'objet FLAML et celui de l'estimateur extrait sont tous deux 0.8561. L'égalité de toutes les prédictions de test est vérifiée avant l'enregistrement.

Run AutoML : 6f7f9441bde44326bae1aa580bfb2558
Run manuel : 0e5adfd53b684e42b31f00625b2a3cd2

Capture 1a - Run automl-90s dans MLflow

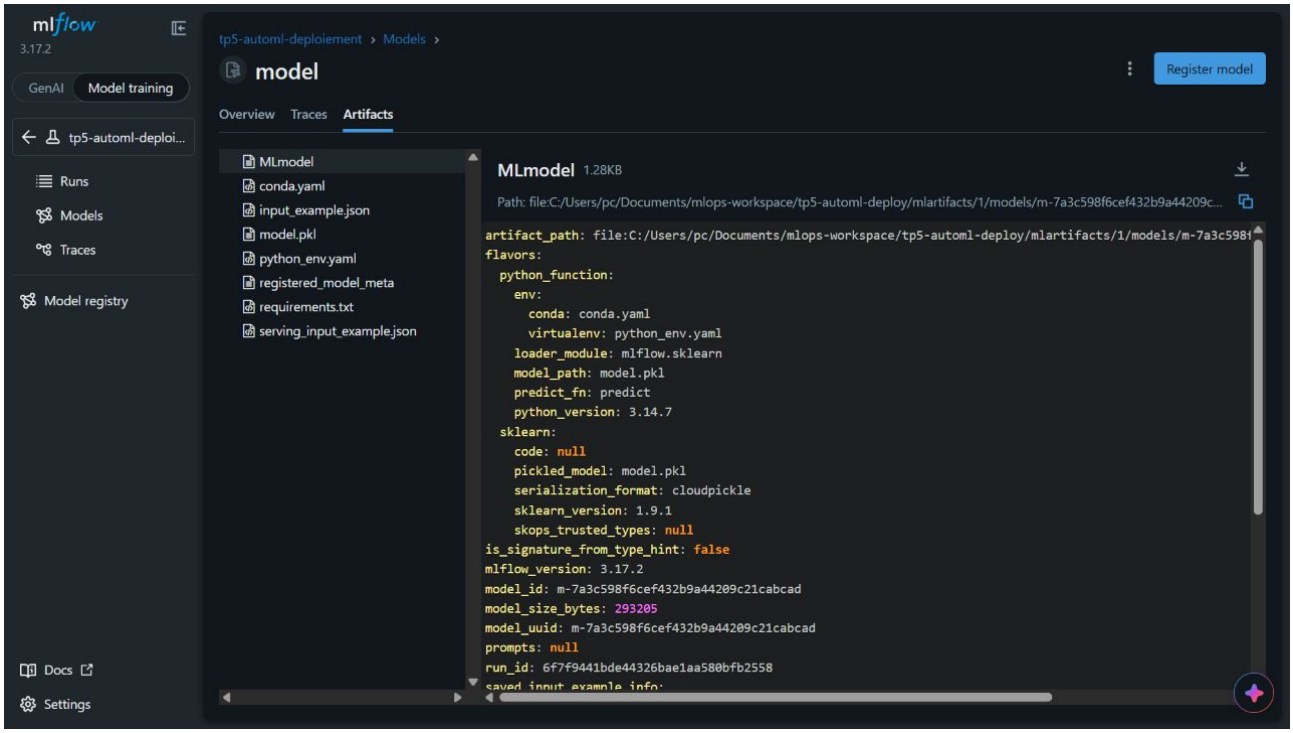


Capture 1b - Famille et réglages gagnants

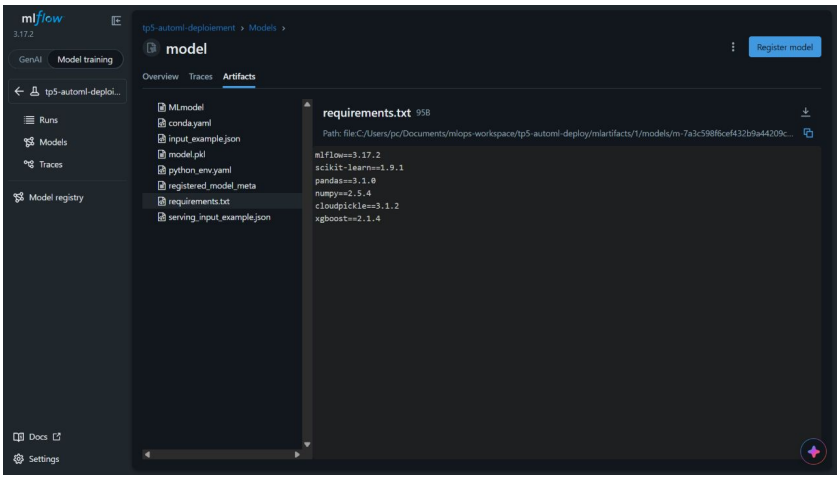


2.1. Packaging du modèle AutoML

Capture 1c - Fichier MLmodel et dépendances du gagnant



Dépendances enregistrées dans MLflow



Formats et dépendances vérifiés

AutoML : cloudpickle ; Python 3.14.7 ; MLflow 3.17.2 ; scikit-learn 1.9.1 ; XGBoost 2.1.4 ; pandas 3.1.0 ; NumPy 2.5.4. Le modèle manuel est enregistré en skops. La signature attend montant en double et les trois autres variables en entiers longs.

Les fichiers originaux MLmodel et requirements.txt des deux modèles sont conservés dans reports/artefacts/. Les flavors python_function et sklearn permettent le chargement et le service HTTP.

3. Promotion et services HTTP

Les modèles fraude-automl et fraude-manuel sont enregistrés en version 2 et 1. Le script promouvoir.py attribue l'alias production à la dernière version READY de chacun. Les services chargent models:/fraude-automl@production et models:/fraude-manuel@production.

Capture 2 - Journaux des services et contrôles HTTP

Preuve 2 - Services MLflow demarres

Transcription des journaux reels .runtime/*.err.log et de verification.json

AUTOML - 127.0.0.1:8100

```
2026/11/02 11:53:14 INFO mlflow.pyfunc.backend: === Running command 'uvicorn --host
127.0.0.1 --port 8100 --workers 1 mlflow.pyfunc.scoring_server.app:app'
INFO:     Started server process [12132]
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8100 (Press CTRL+C to quit)
```

MANUEL - 127.0.0.1:8101

```
2026/11/02 11:53:37 INFO mlflow.pyfunc.backend: === Running command 'uvicorn --host
127.0.0.1 --port 8101 --workers 1 mlflow.pyfunc.scoring_server.app:app'
INFO:     Started server process [19676]
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8101 (Press CTRL+C to quit)
```

Controles du script src/verifier.py

```
automl: /health HTTP 200, alias v2 READY
  Rechargement identique sur 1 250 lignes : OK
  Predictions HTTP identiques sur 30 lignes : OK
manuel: /health HTTP 200, alias v1 READY
  Rechargement identique sur 1 250 lignes : OK
  Predictions HTTP identiques sur 30 lignes : OK
```

Cette figure est une transcription des journaux réels, pas une capture de fenêtres de terminaux. Les fichiers .runtime/automl.err.log et manuel.err.log conservent les sorties originales.

Les deux /health répondent HTTP 200. Les modèles rechargés depuis les alias reproduisent toutes les prédictions du test local. Pour chaque service, un lot de 30 transactions envoyé par HTTP reproduit aussi les prédictions locales.

L'alias est résolu au chargement : modifier production ne remplace pas le modèle déjà en mémoire. Un redémarrage du service est nécessaire après promotion ou retour à une ancienne version.

4. Client, latences et requête invalide

Le client envoie montant=8500.0, anciennete=3, freq_24h=9 et nuit=1. Chaque service renvoie HTTP 200 et predictions=[1]. Chaque répétition comprend 10 appels d'échauffement non mesurés et 200 appels chronométrés avec perf_counter ; les réponses sont validées.

Capture 3 - Résultats du client et rejet de la requête invalide

Preuve 3 - Client HTTP et signature

Transcription des mesures JSON ; aucune latence ni reponse simulee.

Client - port 8100

```
reponse : 200 {'predictions': [1]}
Essai 1 : mediane 31.21 ms | p95 41.16 ms
Essai 2 : mediane 30.34 ms | p95 38.72 ms
Par essai : 10 echauffements + 200 appels mesures et valides.
```

Client - port 8101

```
reponse : 200 {'predictions': [1]}
Essai 1 : mediane 92.13 ms | p95 150.68 ms
Essai 2 : mediane 118.10 ms | p95 300.82 ms
Par essai : 10 echauffements + 200 appels mesures et valides.
```

Requete invalide - HTTP 400


```
{
  "error_code": "INVALID_PARAMETER_VALUE",
  "message": "Failed to predict data '   montant\n0   8500.0'. \nError: Failed to
enforce schema of data '   montant\n0   8500.0' with schema '['montant': double
(required), 'anciennete': long (required), 'freq_24h': long (required), 'nuit': long
(required)]'. Error: Model is missing inputs ['anciennete', 'freq_24h', 'nuit'].",
  "sqlstate": "KAM01",
  "error_class": "SCHEMA_ENFORCEMENT_FAILED"
}
```


La requête contenant seulement montant est rejetée avec HTTP 400 : anciennete, freq_24h et nuit sont manquantes. Il s'agit d'une validation de schéma, pas d'une validation métier de la transaction.

Les mesures incluent la connexion locale, JSON, le contrôle de schéma et l'inférence ; elles n'isolent pas le temps du modèle. Les appels sont séquentiels, sans session persistante ni charge concurrente. Le p95 est calculé avec statistics.quantiles(n=20), comme dans le sujet. Aucune latence cloud n'a été mesurée.

5. Réponses aux questions 1 à 3

Question 1 - Quel modèle mettre en production ?

Je retiendrais AutoML (XGBoost) pour ce TP : le modèle est nettement plus compact et plus rapide, avec une baisse de F1 de 0,0109 et une fraude supplémentaire manquée sur ce test. Le manuel reste préférable si ce coût de détection domine. Les importances globales sont accessibles pour les deux ensembles d'arbres ; une explication locale peut être étudiée avec SHAP. Une importance ne démontre pas un lien causal.

La matrice de confusion est fournie dans les résultats JSON (ordre : [[vrais négatifs, faux positifs], [faux négatifs, vrais positifs]]). Un jeu synthétique et un unique découpage ne suffisent pas à garantir une performance opérationnelle ; il faut fixer un coût des fraudes manquées et un objectif de latence.

Question 2 - Coût mensuel d'un point de terminaison cloud

Consultation : **02/11/2026**. Hypothèse illustrative : une machine n1-standard-2 dans us-central1, à la demande, sans remise, un modèle déployé en continu. Le tarif officiel de prédiction consulté est **0,1095 USD par heure**. Pour 30 jours : 30 × 24 × 0,1095 = **78,84 USD** ; pour 730 heures : **79,94 USD**.

Une machine active reste facturée sans requêtes. Estimation hors entraînement, stockage, réseau, taxes et explications supplémentaires ; elle n'est pas un devis pour europe-west1. La page précise que l'inférence AutoML tabulaire suit les tarifs des modèles personnalisés. Aucun déploiement facturé n'a été lancé. [S1]

Question 3 - Pourquoi le cloud est généralement plus lent ?

Sur ce poste, les médianes AutoML sont 31.21 et 30.34 ms ; celles du manuel sont 92.13 et 118.10 ms. En cloud s'ajoutent le trajet réseau jusqu'à la région, TLS, l'authentification, le routage et d'éventuelles files d'attente. Ces surcoûts expliquent une latence souvent supérieure, sans permettre de déduire un p95 cloud depuis nos mesures locales.

Une machine distante plus puissante peut réduire l'inférence ; la comparaison nécessite un client, une région, une charge et une taille de lot explicitement fixés.

6. Réponses aux questions 4 à 6

Question 4 - Le moteur AutoML change de version

Un nouvel entraînement peut explorer d'autres modèles ou réglages, produire des scores différents et modifier les dépendances. Une évolution du runtime peut aussi affecter un modèle déjà servi. Il faut conserver les versions du moteur et des bibliothèques, le jeu de données et son empreinte, la configuration gagnante, l'artefact et la signature. Une validation comparative et une possibilité de retour arrière précèdent la promotion.

Question 5 - Garantir la réversibilité

Dès le choix d'une plateforme, vérifier les formats d'export et le droit d'utiliser le modèle hors du fournisseur. Exporter, quand cela est possible, le modèle, le prétraitement, la signature et les dépendances ; tester le chargement et l'inférence hors plateforme sur un jeu de référence. Garder les données et un pipeline indépendant pour pouvoir réentraîner si l'export est impossible. Une API seule ne garantit pas la réversibilité.

Question 6 - Pourquoi cloudpickle plutôt que skops ?

L'estimateur choisi par FLAML peut appartenir à LightGBM, XGBoost ou scikit-learn. Dans ce TP, cloudpickle simplifie le packaging via le flavor sklearn, tandis que le Random Forest manuel est explicitement enregistré en skops avec le type sklearn.tree._tree.Tree autorisé. Le format et les dépendances sont visibles dans MLmodel. [S2]

Pickle et cloudpickle peuvent exécuter du code à la désérialisation : ne charger que des artefacts de confiance. L'usage est limité ici à un modèle entraîné localement et conservé dans notre registre. Skops permet de contrôler les types chargés ; une liste de confiance élargie sans examen annule cette protection. [S3]

Traçabilité et état du rendu

Le dépôt local comporte les scripts, les mesures brutes, la comparaison et les dépendances figées. data/, models/, .venv/, mlflow.db et mlartifacts/ sont exclus de Git. Le README donne les commandes de reproduction et d'arrêt. Les temps humains restent à renseigner. L'URL publique GitHub devra être ajoutée après publication ; aucun courriel n'a été envoyé.

Sources consultées

[S1] Google Cloud, tarification Vertex AI / page actuelle de la plateforme, rubriques Tabular data et Prediction and explanation (us-central1).
cloud.google.com/products/gemini-enterprise-agent-platform/pricing

[S2] MLflow, API mlflow.sklearn, formats et skops_trusted_types.
mlflow.org/docs/latest/api_reference/python_api/mlflow.sklearn.html

[S3] scikit-learn, Model persistence, sécurité et limites de portabilité.
scikit-learn.org/stable/model_persistence.html